========== ========== DATA SCIENCE PERSONAL TRAINING: PROJECT SPECIFICATIONS

PROJECT 1: LINEAR REGRESSION

Scenario: A local real estate agency wants to automate the estimation of house prices to help agents set consistent and competitive listing prices. Your role as a Data Scientist is to build a predictive model that estimates the sale price of a house based on its structural and environmental features.

Dataset: Ames Housing Dataset (Commonly found on Kaggle as "House Prices - Advanced Regression Techniques").

Project Instructions & Guidelines:

1. Data Exploration (EDA):
    - Analyze the distribution of the target variable (SalePrice).
    - Investigate correlations between numerical features and the house price.
    - Generate a correlation matrix/heatmap to identify the strongest predictors.
2. Feature Engineering & Categorical Encoding (Crucial Step): Carefully examine the 'data_description.txt' file to separate and encode your features properly:

    A. Binary Features (0 or 1):
    - Convert variables with only two logical states into binary format.
    - Examples: 'Street' (Grvl=0, Pave=1), 'CentralAir' (N=0, Y=1).
    - For 'Alley', map 'NA' to 0 and any existing alley access (Grvl/Pave) to 1.

    B. Ordinal Features (The Grading System):
    - Identify variables that imply a universal, hierarchical scale of quality or condition.
    - Map these text values to progressive integers (e.g., NA=0, Po=1, Fa=2, TA=3, Gd=4, Ex=5).
    - Apply this to: 'ExterQual', 'ExterCond', 'BsmtQual', 'BsmtCond', 'HeatingQC', 'KitchenQual', 'FireplaceQu', 'GarageQual', 'GarageCond', 'PoolQC'.
    - Map structural ordinal variables similarly: 'LotShape' (IR3=0 to Reg=3), 'LandSlope' (Sev=0 to Gtl=2), and 'GarageFinish' (NA=0 to Fin=3).

    C. Nominal Features (One-Hot Encoding):
    - Identify purely descriptive variables that do not possess an inherent numerical order (where one option is not universally "better" than another, just different).
    - Examples: 'Neighborhood', 'RoofStyle', 'RoofMatl', 'Foundation', 'BldgType', 'MSZoning'.

- Use One-Hot Encoding ('pd.get_dummies' with 'drop_first=True') to convert these categories into separate 0/1 columns, avoiding the dummy variable trap.

3. Model Training:

- Handle missing values (NaN) appropriately (e.g., fill with 0/NA for missing features like garages/basements, or use median imputation for continuous variables like 'LotFrontage').
- Split your data into an Training set (80%) and a Test set (20%) using a fixed random state.
- Train a standard Linear Regression model.

4. Evaluation:

- Predictions should be evaluated on the Test set.
- Calculate and interpret the R-squared (R^2) score.
- Calculate the Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) to quantify the average dollar amount your model deviates from the real prices.

PROJECT 2: CLASSIFICATION (LOGISTIC REGRESSION)

Scenario: An online banking institution is experiencing an increasing rate of customer churn (customers closing their accounts). Because acquiring new customers is highly expensive, the bank wants to anticipate departures. Your task is to build a classification model to identify high-risk customers, allowing the marketing team to target them with retention offers.

Dataset: Bank Customer Churn Dataset (Available on Kaggle).

Project Instructions & Guidelines:

1. Exploratory Data Analysis & Class Imbalance:

- Check the proportion of customers who left (Churn = 1) versus those who stayed (Churn = 0).
- Document the class imbalance, as churn datasets are typically heavily skewed towards customers staying.

2. Data Preparation:

- Feature Scaling: Since standard Logistic Regression is sensitive to the scale of numerical inputs, standardize or normalize continuous variables (such as 'Balance', 'Age', or 'CreditScore') using StandardScaler.
- Encode any categorical variables (e.g., 'Gender' or 'Geography') using appropriate encoding techniques (Binary or One-Hot Encoding).

3. Model Training:

- Split the dataset into Training (80%) and Test (20%) sets.
- Train a Logistic Regression classifier.
- (Optional Advanced Step): Adjust the 'class_weight' parameter to 'balanced' if the class imbalance heavily negatively impacts your model's performance.

4. Evaluation:

- Do not rely solely on global Accuracy.
- Generate and display the Confusion Matrix.
- Compute Precision, Recall, and the F1-Score.
- Focus specifically on maximizing 'Recall', as the bank's priority is to detect as many potential churning customers as possible, even if it means including a few false alarms.

========== ========== END OF SPECIFICATIONS - SUBMIT YOUR CODE AND METRICS FOR REVIEW WHEN COMPLETED.
